

Final Paper

I.Title

Canteen Card System

II. Problem Statement

The reliance on cash transactions in school canteens shows significant problems like long wait times during break or lunch hours, errors made by humans, and the risk of students losing money. Furthermore, parents don't have a clear view of their children's spending habits, while canteen staff struggle to manage student debts due to disorganized records from manual record-keeping. This Canteen Card System fixes these real-world problems by providing a digital system. By removing the need for physical money and having automatic debt history and parental monitoring, this project shows a more secure financial way of transactions and encourages financial responsibility among students.

III. Project Overview And Relevance

The Canteen Card System is a multi-purpose financial management app that is built to align with the school cafeteria operations through distinct Student, Staff, and Parent options. The system uses Python for its logic, complex tasks such as role authentication, balance calculations, and transactions. It uses a strong database structure to maintain records of student balances, detailed transactions in histories, and debt reminders, ensuring data safety across different users. It is more than just a simple record-keeping; the project uses data analysis by allowing users to track spending over time, giving insights into canteen usage. This project serves as a demonstration of how software can transform manual tasks into a digital application.

IV. Objectives

The objective of this Canteen-Card System program is to allow users, such as students, parents, or staff and admins, to check the history and balance of their card at the Cafeteria if they have one via a program, allowing easier access.

Key Points/Objectives:

- Easier Access

- Less time and effort to check transactions and spending
- Consistent reminders whenever the balance is low

V. Methodology

The project was developed using the Python programming language. The system uses a simple database structure stored in JSON format to manage user data such as student information, balance, transactions, and purchases.

The program follows the CRUD process:

- **Create** – Users can register or be added to the system with a unique ID.
- **Read** – The system can display user information such as balance, debt, and transaction history.
- **Update** – Users can deposit money, purchase items, or update their account details.
- **Delete** – Certain records, such as transactions, may be removed if needed.

The system has different user roles:

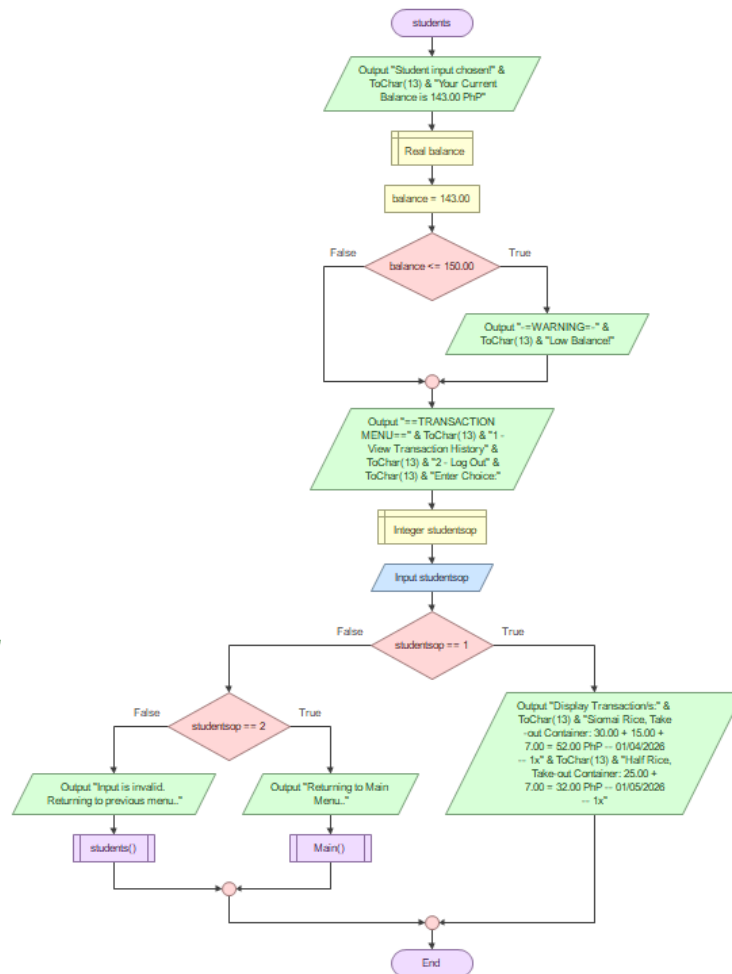
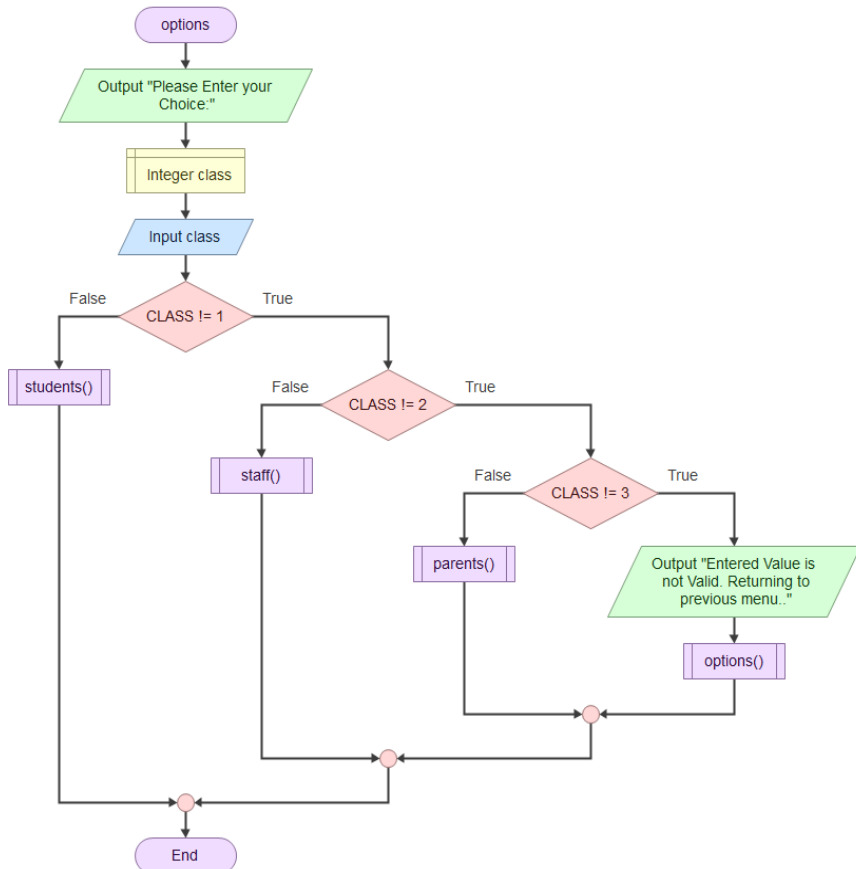
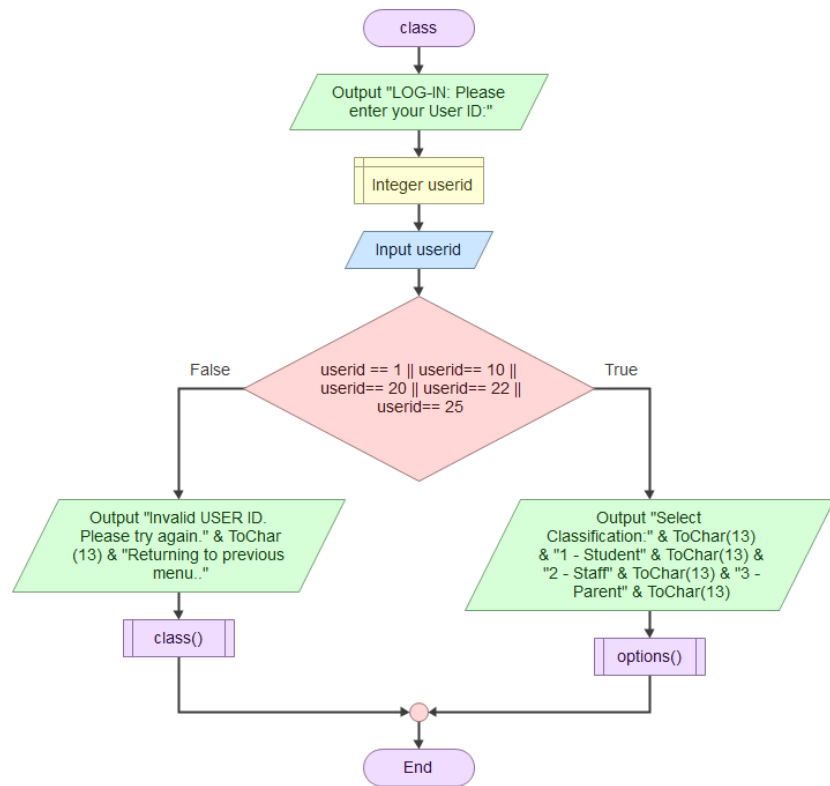
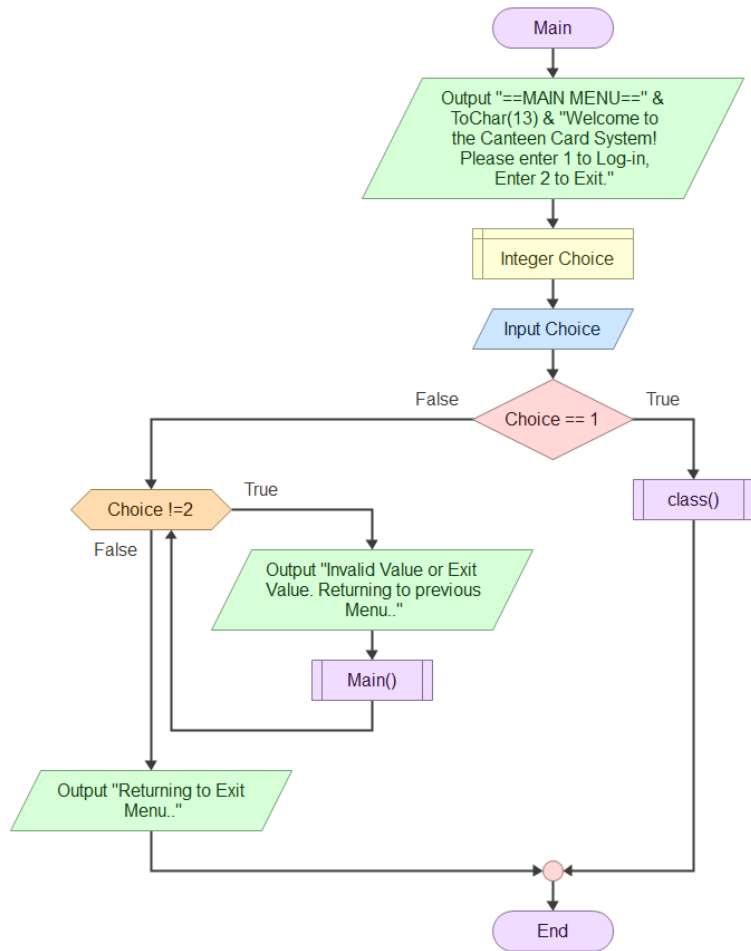
- **Students** – Can buy items, check balance, and view transactions.
- **Staff/Admin** – Has full access to manage all user data.
- **Parent** – Can view transaction history, balance, and debt.

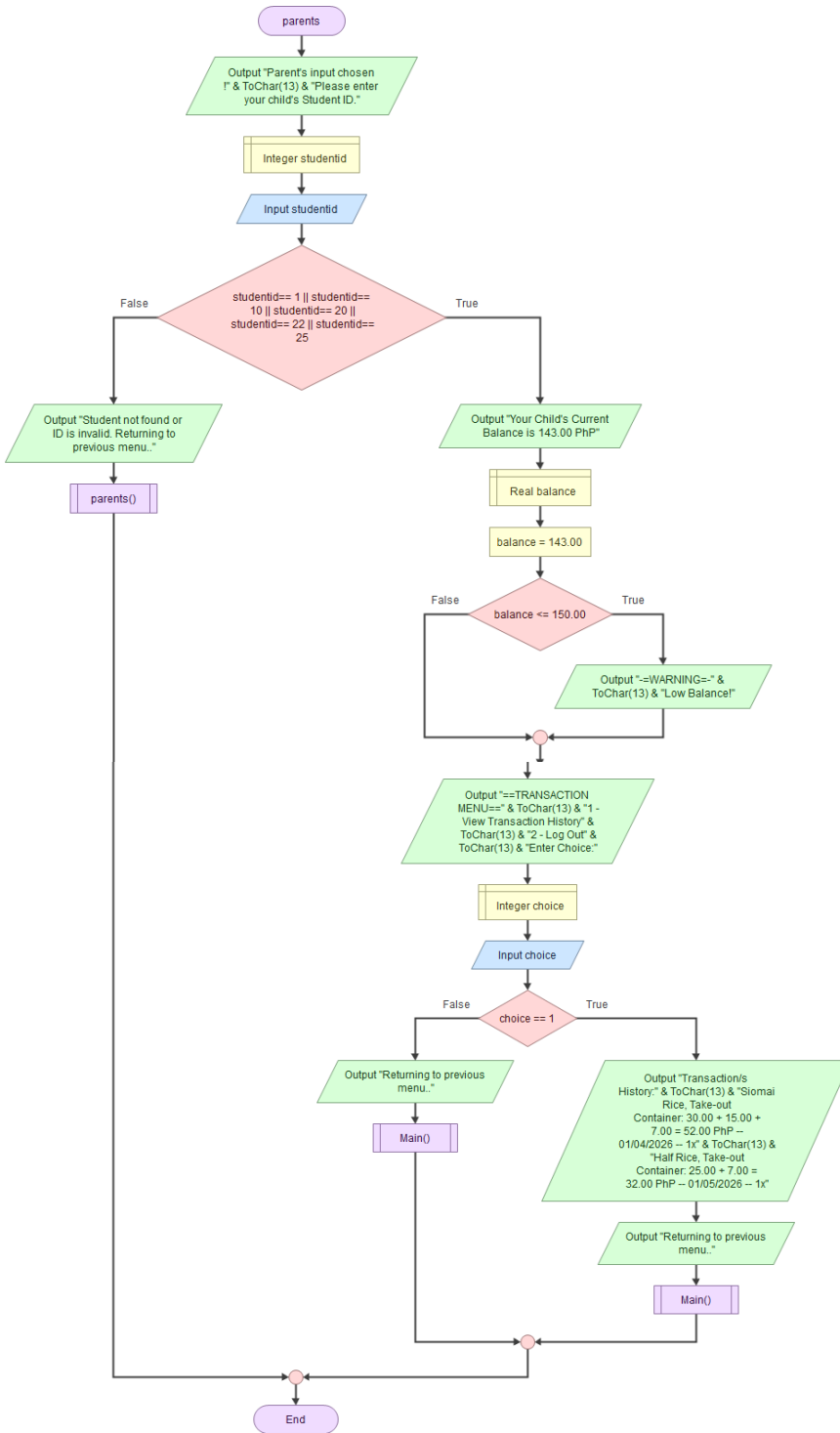
The program flow is controlled using functions and conditional statements. A menu system will guide users to select actions. Input validation is also applied to ensure correct data is entered.

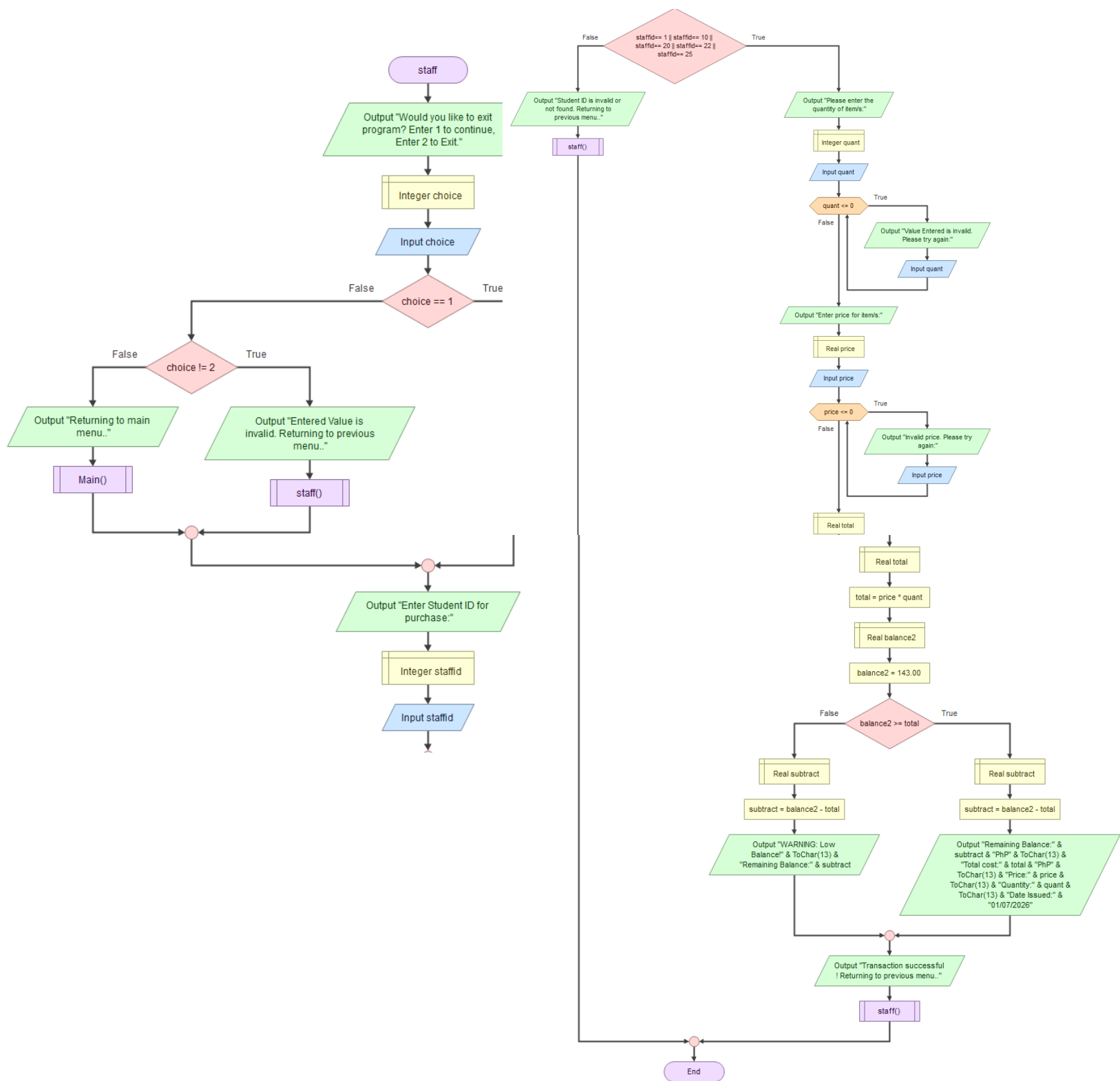
If Flask is used, the system can be turned into a simple web application where users interact through a browser instead of the console.

Basic data analysis may be included by tracking total spending, most purchased items, or user balances to help monitor usage of the system.

VI. Flowchart







VII. Program

VIII. Team Roles

Name	1st Quarter	2nd Quarter	3rd Quarter	4th Quarter
Abadejos, Myles Deric D.C.	Proposed Solution, Problem Decomposition	Flowchart (Students)	Program Code (Students)	None
De Chavez, Kim Venidict B.	Problem Statement, Problem Decomposition	Project Decomposition	Program Code (Parents)	Project Overview /Statement and Conclusion
Manalo, Chelsea Margaret V.	Feasibility and Scope	Project Decomposition, Flowchart	Program Code (Overall + additional features)	PPT, Program
Mundo, Sachi Ysabelle M.	Problem Identification	Flowchart (Parents)	Program Code (Log-in)	Flowchart, Objectives, Readme File
Razon, Xedrick Loise A.	None	None	Program Code (Staff)	Methodology

IX. References